Module 1

Architecture & Design Patterns

MVVM vs MVI

MVVM

- Multiple `StateFlow`s — one per state field
- View calls named ViewModel functions
- Side effects bolted on via `SharedFlow`
- Flexible, easy to adopt incrementally
- State updates are not atomic — inconsistency window between assignments

MVI

- Single immutable `UiState` object
- All interactions via `processIntent(Intent)`
- Side effects are a first-class `Channel<SideEffect>`
- Strict unidirectional data flow
- State updates are atomic — view never sees a half-updated state

MVVM and State Management

MVVM with `ViewModel` and `StateFlow` is the backbone of modern Android development. The ViewModel survives configuration changes and exposes UI state as a stream that composables or views observe. When combined with Unidirectional Data Flow — where events flow up and state flows down — the result is a UI that is predictable, testable, and easy to reason about. In Nami, every screen is driven by a single `UiState` data class emitted from a `StateFlow`, making the full screen state inspectable at any point.